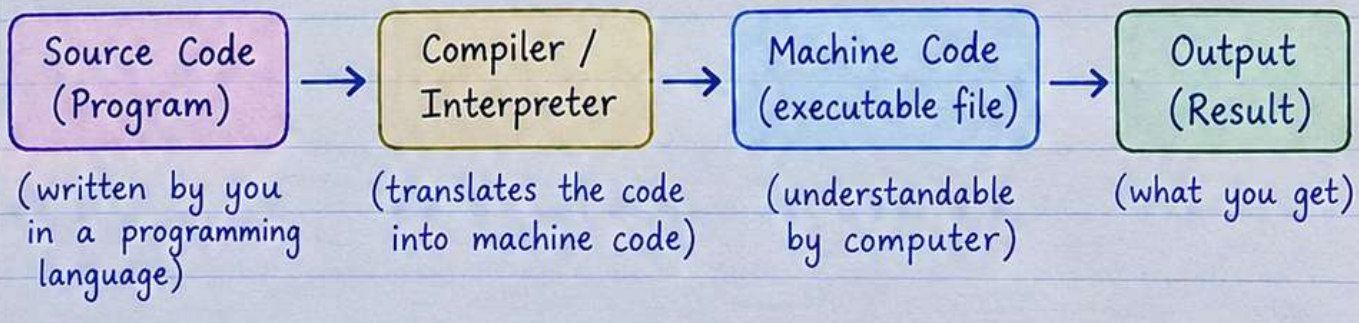
What is a Program ?

Definition :

A program is a set of instructions written in a programming language to perform a specific task.

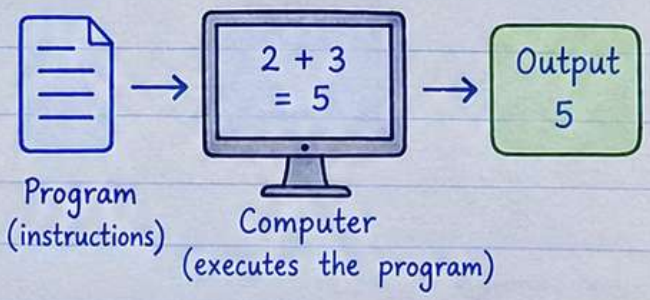
- It tells the computer what to do, step by step.
- A program is basically a solution to a problem, written in a language that a computer can understand.
- Programs are created by humans (developers) to get a desired output.
- Without a program, a computer is just a machine and cannot do anything useful.

How a Program Works ?



Example in Real Life :

Suppose you want to make a calculator app. You write a program with different instructions (like add, subtract, multiply, etc.). The computer reads these instructions, executes them and gives the result.



Key Point :

A program is not just a bunch of code, it is a logical set of instructions that helps the computer to perform a task and produce a result.

What is Software ?

Definition :

- Software is a set of programs, procedures and related data that tells a computer what to do and how to do it.
- It acts as an interface between the user and the hardware.
- Without software, a computer cannot perform any useful task, no matter how powerful its hardware is.

→ In simple words, software is a collection of instructions that makes the computer work.

Types of Software :

Software

1. System Software

(Runs and manages the computer hardware and provides a platform for other software.)

2. Application Software

(Helps the user to perform specific tasks or solve specific problems.)

1. System Software :

System software controls the computer's hardware and provides basic services for application software. It includes:

a) **Operating System :**
- Manages hardware resources (CPU, memory, devices, etc).
- Provides a platform to run other software.

Example : Windows, Linux, macOS, Android.

b) **Device Drivers :**
- Act as a bridge between the operating system and hardware devices.
- Help the OS to communicate with devices.

Example : Printer driver, graphics driver, sound driver.

c) **Language Translator :**
- Converts high-level language code into machine code.
- Includes compilers, interpreters and assemblers.

Example : GCC (compiler), Python (interpreter), Assembler.

d) **Utility Software :**
- Helps in maintaining and optimizing the system.
- Includes antivirus, disk cleanup, backup tools, etc.

Example : Windows Defender, Disk Cleanup, WinRAR.

2. Application Software :

Application software is designed to help users perform specific tasks. It is further divided into two types :

a) **General Purpose Software :**
- Used for common and everyday tasks.
- Available for a wide range of users.
- **Examples :** MS Word, Excel, PowerPoint, Google Chrome, Mozilla Firefox, VLC Media Player, etc.

b) **Specific Purpose Software :**
- Designed for a particular task or field.
- Used by a specific group of users.
- **Examples :** Tally (accounting), Photoshop (image editing), AutoCAD (designing), Hospital Management System, Banking Software, etc.

What is Programming ?

Definition :

- Programming is the compressasive process of the computer largest designing, building, testing and maintaining software application.
- It is the creative and logical art of teaching a computer how to solve a real problem.

In simple words :

" Programming is telling a computer what to do, step by step, so that it can solve a problem."



5 Stages of Programming :

1. Defining the Problem → Understand the problem and find what the program should do.
2. Designing the Dialog → Plan the solution and design the flow of the program (algorithm and flowchart).
3. Coding </> → Write the actual program using a programming language.
4. Testing and Debugging → Check the program for errors and fix the problems (bugs).
5. Maintenance and Updates → Make changes, improvements and updates when needed.

What is Coding ?

Definition :

- Coding is the process of translating human instructions into a language that a computer can understand and execute.
- It is the specific act of writing lines of code using a process set of grammatical rules called **syntax**.

In simple words :

"Coding means writing instructions in a language that a computer understands."

The 3 States of Coding :

1 The Code

(Human Written)

- You write the code using a programming language.
- It is in text form.

→

2 The Translation

(Compiler / Interpreter)

- Converts the code into machine language (0s and 1s).
- Checks for errors (in some cases).

→

3 The Result

(Output)

- The computer executes the machine code.
- We get the final output (result).

Example :

Let's take a simple program to print "Hello, World!".

Code (Human Written)

```
1  print("Hello, World!");
```

(Written by the programmer using a programming language)

→

Translation
(Compiler / Interpreter)



Converts the code into machine language and checks for errors.

→

Result (Output)

```
Hello, World!
```

(Displayed on the screen or output device)

* Summary :

- Coding = writing instructions (code).
- Syntax = rules for writing code.
- 3 states = Code → Translation → Result.

Programming VS Coding

1. What is Programming ?

Programming is the process of planning, designing, building, testing and maintaining software applications.

It is the logical and creative process of solving real-world problems using a computer.

Programming is the bigger picture...

Plan & Design → Write Code → Test & Debug → Maintain & Update

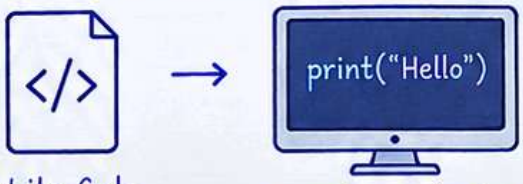
Complete process of building software

2. What is Coding ?

Coding is the process of writing instructions in a programming language that a computer can understand and execute.

It is the actual act of writing lines of code using a set of rules called syntax.

Coding is a part of programming...



Write Code (using syntax) → Computer understands and executes

Key Differences

Point	Programming	Coding
Meaning	Overall process of creating software applications.	Writing instructions in a programming language.
Scope	Includes planning, designing, coding, testing, maintenance, etc.	Only writing the actual code.
Focus	Solving a problem and building a complete solution.	Converting logic into code using syntax.
Skills Needed	Logical thinking, problem solving, planning, design, etc.	Knowledge of programming languages and syntax.
Output	A complete software application (or system).	Source code (set of instructions).
Example	Building a website, app, game, or software.	Writing HTML, Python, Java, C++ code, etc.

In Short :

- Programming is the complete process of creating software.
- Coding is a part of programming, where we write instructions.
- You can think of programming as the plan, and coding as the language used to bring that plan to life.

Good Programmers Think... Good Coders Write !

Programming Language

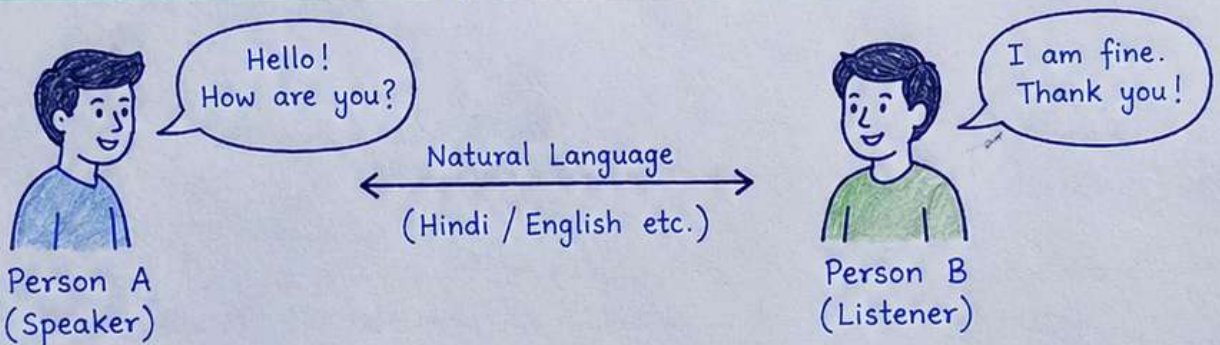
What is Programming Language ?

A programming language is a set of rules and instructions used to communicate with a computer. It helps us to write programs, give commands to the computer and make it perform specific tasks.

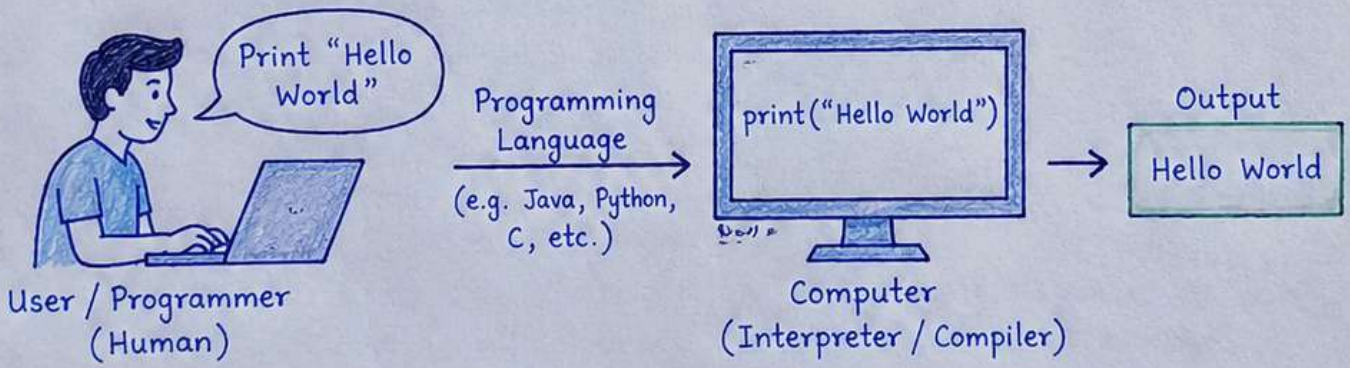
Just like we use a natural language (such as Hindi or English) to talk to humans, we use a programming language to communicate with a computer.

Diagram

①. Human to Human Talking (Natural Language)



②. Human to Computer (Programming Language)



In Short :

- A programming language is a bridge between human and computer.
- It converts our instructions into a form that computer can understand and execute.
- Examples of programming languages : Python, Java, C, C++, JavaScript, etc.

Types of Programming Language

Based on Level of Abstraction

Programming languages can also be classified based on the level of abstraction they provide. This means how close the language is to the machine (computer) and how easy it is to understand and use.

Types of Programming Language

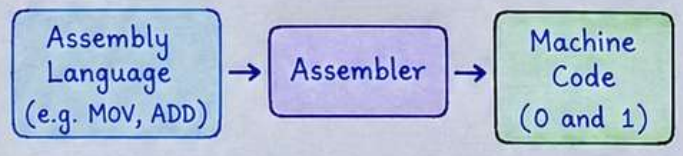
- **Low Level Language** (Closer to Machine)
- **High Level Language** (Closer to Human)

1. Low Level Language

- These languages are closer to the machine and are written in binary form (0 and 1) or in simple machine instructions.
- They are hardware dependent (specific to a particular machine).
- They are difficult to learn and write.

Examples : Machine Language, Assembly Language.

Diagram :



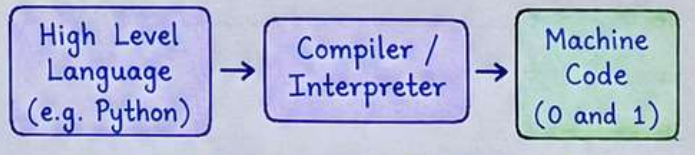
(Low level language → Machine code)

2. High Level Language

- These languages are closer to human language and are easy to understand and use.
- They are machine independent (can run on different systems with little or no change).
- They are more user-friendly and require less effort to write and maintain.

Examples : Python, Java, C, C++, JavaScript, PHP, etc.

Diagram :



(High level language → Machine code)

Key Difference :

Low Level Language	High Level Language
• Closer to machine	• Closer to human
• Binary / simple instructions	• English-like syntax
• Hardware dependent	• Machine independent
• Difficult to learn	• Easy to learn
• Example: Machine language, Assembly	• Example: Python, Java, C, C++, etc.

Types of Programming Language

Based on Execution Method

Programming languages can also be classified based on how the code is executed (i.e. how the instructions are converted into machine code). They are:

1. Compiled Language

- Entire source code is translated into machine code before execution.
- The translated code is stored in a separate file (executable file).
- Execution is fast because no translation is needed during runtime.

Example :

C, C++, Go, Rust

2. Interpreted Language

- Source code is executed line by line at runtime by an interpreter.
- No separate executable file is created.
- Slower than compiled languages but easier to debug and modify.

Example :

Python, JavaScript, PHP, Ruby

3. Hybrid Language

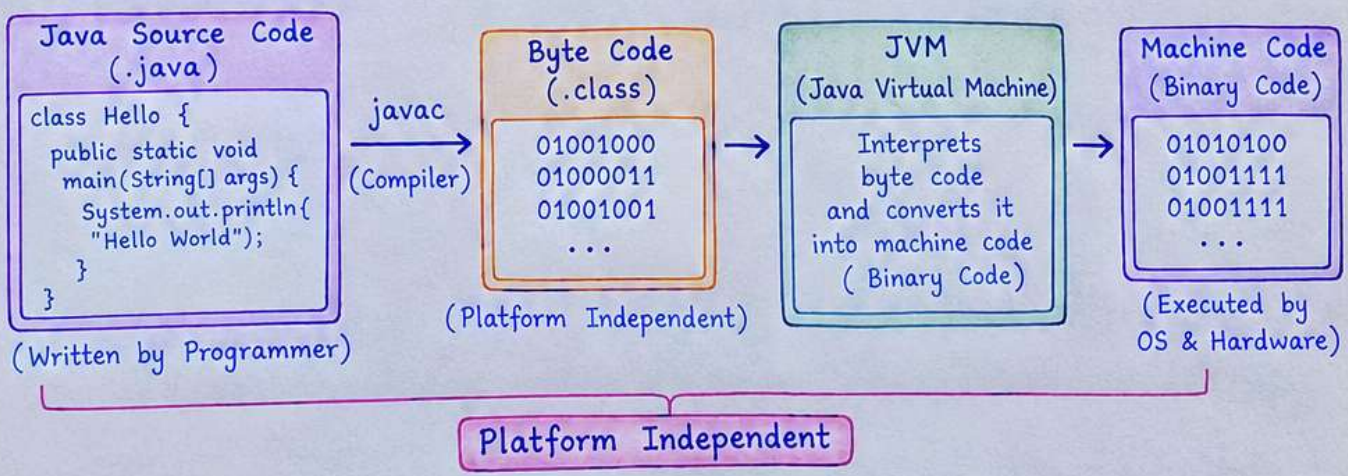
- Combines both compiled and interpreted approaches.
- Source code is first compiled into an intermediate form (like byte code), which is then interpreted or executed by a virtual machine.
- Offers portability and good performance.

Example :

Java (JVM), C# (CLR)

Example : Java (Hybrid Language)

In Java, the source code is first compiled into byte code. Then, the Java Virtual Machine (JVM) interprets the byte code into machine code (binary code) at runtime. This makes Java platform independent.



Key Points :

- Compiled → 1 time translation, fast execution (e.g. C, C++).
- Interpreted → line by line execution (e.g. Python, JavaScript).
- Hybrid → both compiled + interpreted (e.g. Java, C#).
- Java → source code → compiled (javac) → byte code → JVM → interpreted → binary code.
- JVM makes Java platform independent (runs on any OS with JVM).

Types of Programming Language

3. Based on Programming Paradigm

Programming languages can also be classified based on the programming paradigm they follow. A paradigm defines the way we think and structure a program.

1. Procedural Programming Language

What is it?

- It is based on a set of procederes or functions that operate on data. The program is divided into small procedures and executed in a sequence.

Key Features

- Follows step-by-step approach.
- Uses functions / procedures.
- Data and functions are separate.
- Easy to understand and debug for small programs.

Example (C)

```c
#include<stdio.h>
void main() {
    printf("Hello World");
}
```



(Step by step execution)

2. Functional Programming Language

What is it?

- It is based on mathematical functions. It avoids changing the state and uses pure functions (no side effects).

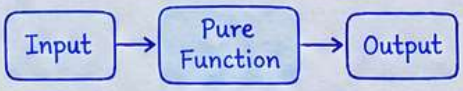
Key Features

- Uses functions as first-class citizens.
- No mutable data (immutable).
- Supports recursion.
- Produces the same output for the same input.

Example (Python)

```python
def square(n):
    return n * n

num = 5
print(square(num))
```



(No change in data)

3. Object Oriented Programming Language

What is it

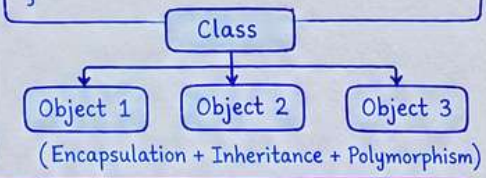
- It is based on the concept of objects. It organizes the program around data (objects) and the methods (functions) that operate on that data.

Key Features

- Uses classes and objects.
- Supports encapsulation, inheritance and polymorphism.
- Better code organization and reusability.
- More suitable for large and complex applications.

Example (Java)

```java
class Car {
    void display() {
        System.out.println("Car");
    }
}
public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.display();
    }
}
```



(Encapsulation + Inheritance + Polymorphism)

*** Note : A Programming Language can use Multiple Paradigms**

- Many modern programming languages support more than one paradigm.
  For example:
  - → Python : Procedural + Functional + Object Oriented
  - → Java : Object Oriented + Procedural (also supports functional features)
  - → C++ : Procedural + Object Oriented + Generic Programming (template-based)

Types of Programming Language
(Based on Type Checking)

➤ Based on type checking, programming languages are mainly divided into two types :

① Statically Typed (Static Type Checking)
② Dynamically Typed (Dynamic Type Checking)

1. Static Type Checking

- In static type checking, the data type of a variable is checked at compile time (before the program runs).
- The type of a variable cannot be changed once it is declared.
- It helps to catch type errors early.

Example : C, C++, Java, Go (these are statically typed languages)

Variable example :

```
int a = 10;          // a is an integer
float b = 5.5;       // b is a float
char c = 'A';        // c is a character
```

2. Dynamic Type Checking

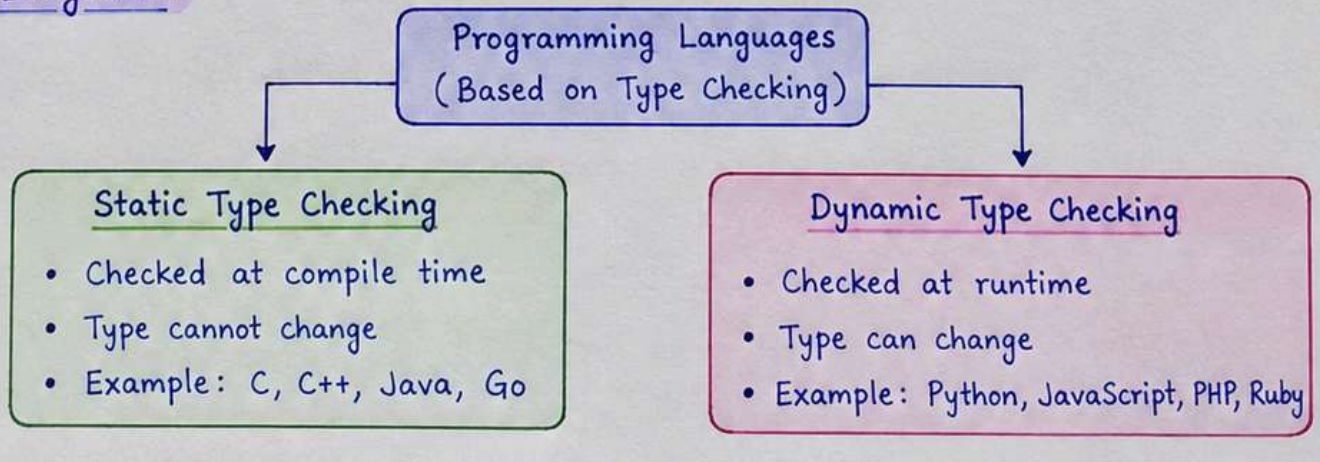
- In dynamic type checking, the data type is checked at runtime (during execution).
- The type of a variable can be changed during the program.
- It gives more flexibility, but may lead to runtime errors.

Example : Python, JavaScript, PHP, Ruby (these are dynamically typed languages)

Variable example :

```
x = 10          # initially integer
x = "Hello"     # now x is a string
x = 3.14        # now x is a float
```

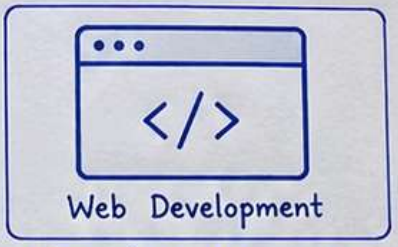
Diagram :



Types of Programming Language (Based on Application Area)

1 Web Development Languages

- These languages are used to create websites and web applications that run in a browser.
- They help in building the structure, design and interactive features of a website.
- Examples: HTML, CSS, JavaScript, PHP, ASP.NET



Examples: HTML, CSS, JavaScript, PHP, ASP.NET

2 Mobile Development Languages

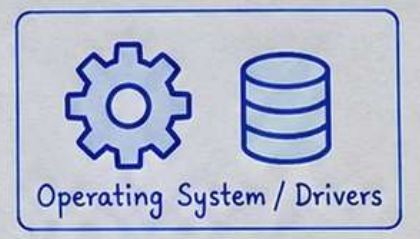
- These languages are used to develop applications for mobile devices like Android and iOS.
- They help in creating apps that run on smartphones and tablets.
- Examples: Java (Android), Kotlin (Android), Swift (iOS), Dart (Flutter)



Examples: Java, Kotlin, Swift, Dart

3 System Programming Languages

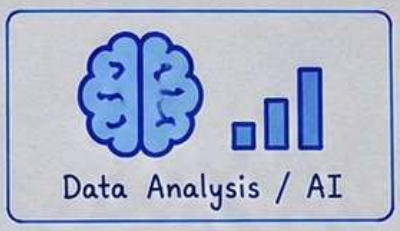
- These languages are used to develop system software like operating systems, compilers, drivers and utilities.
- They work close to the hardware and provide basic functionality to the computer.
- Examples: C, C++, Assembly, Rust



Examples: C, C++, Assembly, Rust

4 Data Science and AI Languages

- These languages are used for data analysis, machine learning, artificial intelligence and scientific computing.
- They provide powerful libraries and tools for working with data and building intelligent systems.
- Examples: Python, R, Julia, MATLAB



Examples: Python, R, Julia, MATLAB

5 Game Development Languages

- These languages are used to create video games and interactive 2D/3D graphics.
- They provide tools and libraries for game logic, graphics, physics and animation.
- Examples: C#, C++, Java, Python (Pygame), JavaScript (Three.js)



Examples: C#, C++, Java, Python (Pygame), JavaScript (Three.js)